ML Lab Week 14: CNN Image Classification

Rock, Paper, Scissors | SRN: PES2UG24CS616

1. Introduction

The objective of this lab was to design, build, train and evaluate a Convolutional Neural Network (CNN) in PyTorch that classifies images of hand gestures into three classes: rock, paper and scissors. The Rock Paper Scissors dataset from Kaggle (2,188 images) was loaded with `ImageFolder`, split 80/20 into training and test sets, and used to train a three-block CNN followed by a fully-connected classifier. The trained model was evaluated on unseen test images and then used to play a simple rock-paper-scissors game against itself.

2. Model Architecture

Input images are resized to 128x128, converted to tensors and normalized with mean 0.5 and standard deviation 0.5 per channel. The network has two parts: a convolutional feature extractor and a fully-connected classifier.

Convolutional block (3 stages; every convolution uses kernel size 3x3, stride 1, padding 1, so only the max pooling changes spatial size):

Stage	Layers	Output shape
Input	RGB image	3 x 128 x 128
1	Conv2d(3 to 16) + ReLU + MaxPool2d(2)	16 x 64 x 64
2	Conv2d(16 to 32) + ReLU + MaxPool2d(2)	32 x 32 x 32
3	Conv2d(32 to 64) + ReLU + MaxPool2d(2)	64 x 16 x 16

Fully-connected classifier:

- Flatten: 64 x 16 x 16 = 16,384 features
- Linear(16384 to 256) followed by ReLU
- Dropout with p = 0.3 for regularization
- Linear(256 to 3), giving one logit per class (paper, rock, scissors)

Max pooling halves the spatial size at each stage (128 to 64 to 32 to 16) while the channel count doubles, so the network learns progressively richer features at lower resolution and keeps the parameter count manageable.

3. Training and Performance

Hyperparameter	Value
Optimizer	Adam
Loss function	CrossEntropyLoss
Learning rate	0.001
Epochs	10
Batch size	32

Train / test split	80% / 20% (1,750 / 438 images)
Device	CPU

The average training loss fell steadily from 0.6298 in epoch 1 to 0.0089 in epoch 10, reaching its lowest value of 0.0025 at epoch 9:

Epoch	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Loss	0.6298	0.1860	0.0908	0.0553	0.0371	0.0328	0.0297	0.0072	0.0025	0.0089

Final Test Accuracy: 95.43% on the 438 held-out test images.

The model also correctly predicted the single-image test (a paper image was classified as paper), and in the game demo it classified a scissors image and a paper image correctly, declaring scissors the winner.

4. Conclusion and Analysis

Results. The CNN performed well: 95.43% test accuracy on a three-class problem after only 10 epochs on a CPU. The very low final training loss compared with the remaining test error suggests mild overfitting, which is expected for a small dataset and a model with a large dense layer (16,384 x 256 weights). Dropout helped but did not remove the gap completely.

Challenges. The training loss was not perfectly monotonic (0.0025 at epoch 9 rose to 0.0089 at epoch 10), which is normal noise from mini-batch Adam updates. Because `random_split` was used without a fixed seed, the exact accuracy varies slightly from run to run. Training on CPU was also slower than it would be on a GPU.

Possible improvements:

- **Data augmentation** (random flips, rotations, brightness and zoom changes) to improve generalization and reduce overfitting.
- **Deeper or regularized architecture:** add Batch Normalization and an extra convolutional layer, or use transfer learning with a pretrained network such as ResNet18 or MobileNet, to push accuracy higher.